

Agenda Personal de Tareas

Entrega final — Versión JavaFX (v1.1)

Autor: Juan Luis Iglesias Llorena

Asignatura: Entornos de Desarrollo

Tipo de proyecto: Aplicación de gestión y utilidad personal

Tecnologías: Java 17, JavaFX 21, Maven, JUnit 5

Este documento recoge la entrega final del proyecto de la asignatura Entornos de Desarrollo. Incluye el diagrama de clases definitivo, el manual de usuario, las credenciales de acceso y un resumen de los cambios respecto a la propuesta y al análisis originales. La revisión **v1.1** incorpora dos cambios funcionales solicitados durante la demo: el aislamiento de tareas por usuario autenticado y el botón de cierre de sesión que vuelve al login sin reiniciar la aplicación.

Índice

1. Introducción
2. Diagrama de clases final
3. Manual de usuario
 - 3.1. Requisitos previos
 - 3.2. Ejecución del proyecto
 - 3.3. Inicio de sesión y registro de usuarios
 - 3.4. Funcionalidades disponibles
 - 3.5. Tareas privadas por usuario
 - 3.6. Cierre de sesión
 - 3.7. Persistencia de datos
4. Cambios respecto a la versión original
 - 4.1. Funcionalidades planificadas omitidas
 - 4.2. Funcionalidades nuevas añadidas
5. Pruebas unitarias y verificación
6. Estructura del proyecto

1. Introducción

La Agenda Personal de Tareas es una aplicación de escritorio que permite al usuario gestionar de manera sencilla sus tareas personales: crear, eliminar, modificar, buscar, filtrar y cambiar el estado de cada tarea. La versión inicial se implementó en consola, tal como se proponía en la Propuesta de proyecto final, y la entrega final incluye, además, una versión con interfaz gráfica construida con JavaFX y persistencia en disco.

El proyecto se estructura siguiendo una arquitectura por capas (modelo, gestor, persistencia, autenticación e interfaz) que respeta el diagrama de clases definido durante la fase de análisis, ampliándolo con las clases necesarias para la interfaz JavaFX y el acceso a fichero.

La revisión **v1.1** incorpora dos cambios funcionales motivados por las pruebas de usuario: el **aislamiento de tareas por usuario autenticado** (cada cuenta sólo ve sus propias tareas) y el **cierre de sesión** sin reiniciar la aplicación, accesible desde la propia ventana principal.

2. Diagrama de clases final

El siguiente diagrama UML resume las clases que componen la entrega final, sus atributos, métodos y las relaciones entre ellas. La revisión v1.1 añade el atributo **owner** a la clase *Task*, el método *getTasksByOwner* en *TaskManager* y un callback de logout entre *MainView* y *LoginView*; estos cambios no se reflejan en la figura para mantener el diagrama compacto, pero sí en el texto y el código.

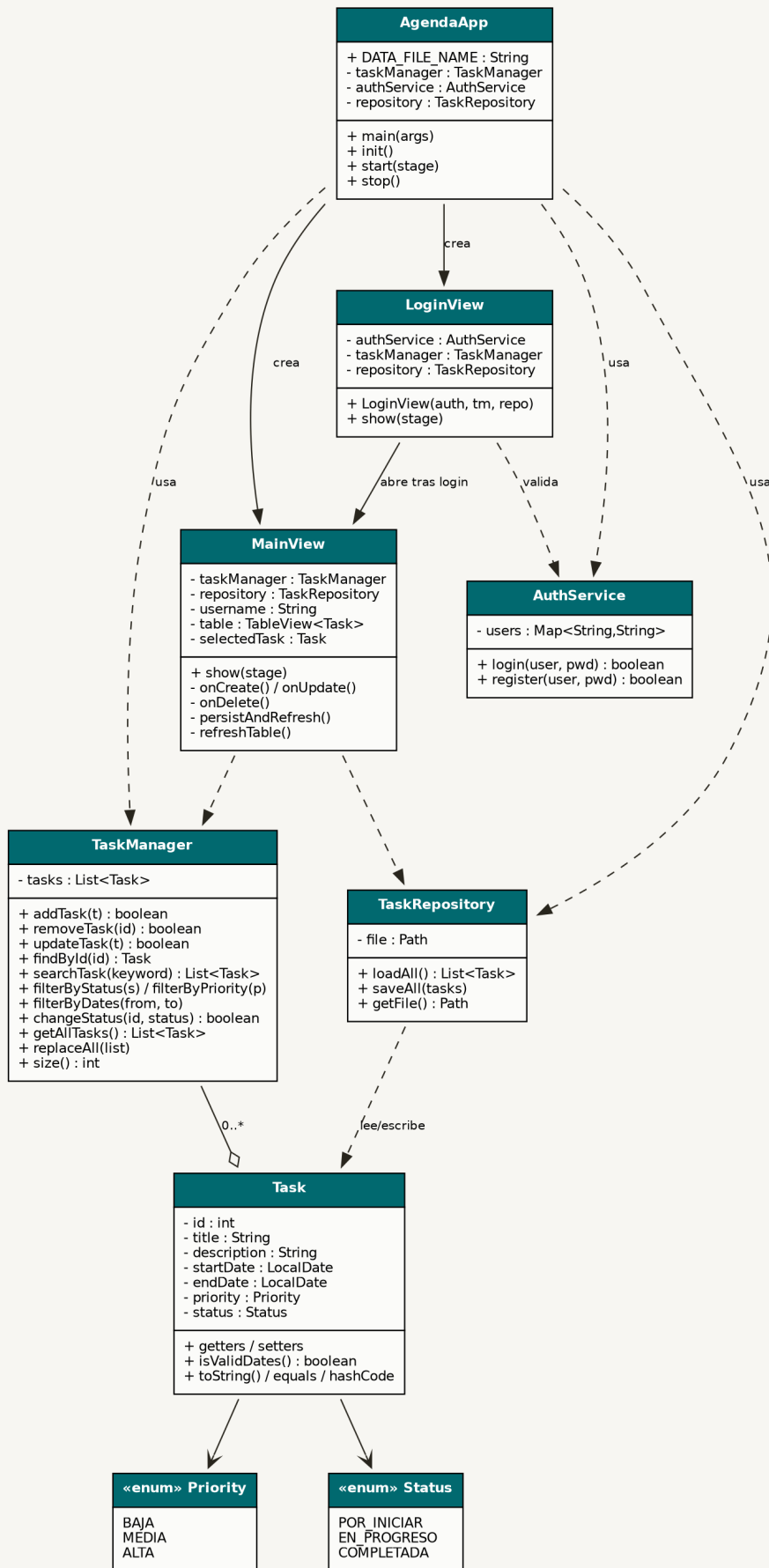


Figura 1. Diagrama de clases de la Agenda Personal de Tareas (versión JavaFX).

Descripción de paquetes y clases

Paquete model — Task, Priority, Status

Modelo de dominio. *Task* representa una tarea con id, título, descripción, fechas, prioridad, estado y, desde la versión 1.1, **owner** (nombre del usuario propietario). *Priority* y *Status* son enumeraciones que describen, respectivamente, la importancia y el estado del ciclo de vida.

Paquete manager — TaskManager

Concentra toda la lógica de negocio: añadir, eliminar, modificar y buscar tareas; filtrarlas por estado, prioridad o rango de fechas; cambiar su estado; y consultar la lista completa o la del usuario actual mediante *getTasksByOwner(username)*. Mantiene internamente un `ArrayList<Task>`.

Paquete persistence — TaskRepository

Se encarga de cargar y guardar las tareas en un fichero CSV con ; como separador y caracteres especiales escapados. Desde v1.1 incluye una octava columna opcional **owner**; los ficheros antiguos sin esa columna se cargan sin error y sus tareas se asignan al administrador por defecto.

Paquete auth — AuthService

Servicio de autenticación con login y registro de usuarios desde la interfaz. Únicamente se siembra la cuenta por defecto **admin / admin**; el resto de personas crean su propia cuenta desde la pantalla de login. Los usuarios se persisten en `~/ .agenda-tareas/users.csv` para que sigan disponibles tras cerrar y volver a abrir la aplicación.

Paquete ui — AgendaApp, LoginView, MainView

AgendaApp es el punto de entrada JavaFX: instancia los servicios, carga los datos del disco y muestra *LoginView*. *LoginView*, además del inicio de sesión, ofrece la acción «Crear usuario» para registrar nuevas cuentas y, tras autenticar al usuario, abre *MainView* pasándole un callback de logout. *MainView* contiene el formulario de edición, la tabla de tareas (filtrada por el usuario autenticado) y los controles de búsqueda y filtrado, e incorpora el botón **Cerrar sesión** que devuelve el control a la pantalla de login sin cerrar la aplicación.

3. Manual de usuario

Esta sección explica paso a paso cómo poner en marcha la aplicación y utilizar todas sus funcionalidades.

3.1. Requisitos previos

- JDK 17 o superior (probado con JDK 17 y JDK 21).
- Apache Maven 3.9 o superior.
- IntelliJ IDEA (Community o Ultimate) con el plugin Maven activado.
- Conexión a Internet la primera vez, para que Maven descargue las dependencias (JavaFX 21 y JUnit 5).

3.2. Ejecución del proyecto

Desde IntelliJ IDEA (recomendado):

- Abrir *File > Open...* y seleccionar la carpeta *agenda-tareas-javafx* (la que contiene el *pom.xml*).
- Esperar a que IntelliJ termine de descargar las dependencias.
- Abrir la clase *com.juaniglesias.agendatareas.Launcher* y pulsar el botón Run junto al método *main*. **No** se debe arrancar *AgendaApp* directamente: la clase *Launcher* evita el error «faltan los componentes de JavaFX runtime».
- Como alternativa puede usarse el panel lateral *Maven > Plugins > javafx > javafx:run*.

Desde la línea de comandos:

```
mvn clean compile
mvn javafx:run
```

3.3. Inicio de sesión y registro de usuarios

Al arrancar la aplicación se muestra una ventana de login que solicita usuario y contraseña. La aplicación incluye una única cuenta sembrada por defecto:

Usuario	Contraseña
admin	admin

Cualquier otra persona debe crear su propio usuario desde la pantalla de login pulsando el enlace «¿No tienes cuenta? Crear usuario». Se abre un diálogo que pide nombre de usuario y contraseña (con confirmación). El sistema valida los datos y avisa mediante alertas si:

- el usuario o la contraseña están vacíos,
- el nombre de usuario ya existe,
- las dos contraseñas no coinciden,
- el nombre de usuario contiene caracteres no permitidos (; o saltos de línea).

Tras crearse, el usuario se persiste automáticamente en `~/ .agenda-tareas/users.csv`, por lo que estará disponible al cerrar y volver a abrir la aplicación. Si las credenciales introducidas son correctas se abre la ventana principal con el nombre del usuario en la cabecera; si son incorrectas se muestra un cuadro de diálogo de error.

3.4. Funcionalidades disponibles

La ventana principal está dividida en dos áreas: un formulario lateral para editar tareas y una tabla central con la lista de tareas **del usuario autenticado**. La cabecera muestra el nombre del usuario en sesión y el botón **Cerrar sesión**. Las operaciones disponibles son:

Acción	Cómo realizarla
Crear tarea	Rellenar el formulario y pulsar Crear. El identificador se asigna automáticamente; la tarea se e
Modificar tarea	Seleccionar una fila de la tabla, ajustar los campos del formulario y pulsar Actualizar. Sólo se p
Eliminar tarea	Seleccionar una fila y pulsar Eliminar. Sólo se permite eliminar tareas propias.
Cambiar estado	Modificar el ChoiceBox Estado y pulsar Actualizar.
Buscar	Escribir un texto y pulsar Buscar. Busca en título y descripción sin distinguir mayúsculas, dent
Filtrar por estado	Elegir un valor de Estado y pulsar Filtrar estado.
Filtrar por prioridad	Elegir un valor de Prioridad y pulsar Filtrar prioridad.
Filtrar por fechas	Indicar Desde y Hasta y pulsar Filtrar fechas.
Ver todas	Pulsar Ver todas para volver a la lista completa del usuario.
Cerrar sesión	Pulsar el botón Cerrar sesión de la cabecera para volver a la pantalla de login sin reiniciar la a

3.5. Tareas privadas por usuario

Desde la revisión v1.1 cada usuario sólo ve sus propias tareas. El mecanismo es transparente:

- Cada tarea persiste con un campo **owner** (octava columna del CSV).
- Al iniciar sesión, la tabla y todos los filtros muestran únicamente las tareas cuyo propietario coincide con el usuario autenticado.
- Las tareas creadas se etiquetan automáticamente con el nombre del usuario actual, sin pedírselo al usuario.
- Por compatibilidad con instalaciones anteriores, las tareas guardadas sin propietario (formato antiguo de 7 columnas) se consideran tareas del administrador y sólo se muestran al iniciar sesión como *admin*.
- Cualquier intento de modificar o eliminar una tarea ajena se rechaza con un mensaje informativo (aunque, en la práctica, las tareas ajenas no son visibles en la tabla).

3.6. Cierre de sesión

La cabecera de la ventana principal contiene el botón **Cerrar sesión**. Al pulsarlo:

- Se guardan en disco las tareas pendientes para no perder cambios.
- Se sustituye la escena actual por la pantalla de login en la misma ventana, **sin cerrar ni reiniciar la aplicación**.
- A continuación es posible iniciar sesión con otro usuario o crear uno nuevo desde el enlace habitual, lo que facilita comprobar el aislamiento de tareas entre cuentas.

3.7. Persistencia de datos

Las tareas se cargan automáticamente al iniciar la aplicación, se guardan tras cada operación de creación, actualización o eliminación, al cerrar sesión y al cerrar la ventana. Los ficheros utilizados son:

~/agenda-tareas/tasks.csv

~/agenda-tareas/users.csv

tasks.csv usa CSV con ; como separador y caracteres especiales escapados (\;, \n, \\\). Cada línea sigue el formato

id;title;description;startDate;endDate;priority;status;owner. La columna *owner* es opcional al cargar (compatibilidad con ficheros antiguos de siete columnas) y obligatoria al guardar.

users.csv contiene una línea por usuario con el formato usuario;contraseña; si el fichero no existe, la primera vez que arranca la aplicación se siembra automáticamente la cuenta **admin / admin**.

4. Cambios respecto a la versión original

La propuesta inicial y el análisis del proyecto contemplaban una aplicación en Java con un único módulo de consola que evolucionaría hacia JavaFX. La entrega final mantiene la lógica de negocio definida y la amplía con varios módulos nuevos. La revisión **v1.1** añade además el aislamiento por usuario y el cierre de sesión. A continuación se detallan los cambios.

4.1. Funcionalidades planificadas omitidas

- **Interfaz de consola:** la versión consola se sustituye por JavaFX en la entrega final. La clase original *Main* queda fuera del proyecto entregado para evitar dos puntos de entrada y porque la rúbrica exige interfaz gráfica.
- **Ordenación personalizada de tareas:** en el análisis se mencionaba la posibilidad de ofrecer ordenaciones definidas por el usuario (por fecha, prioridad, etc.). En la entrega final la tabla JavaFX permite ordenar al pulsar sobre el encabezado de cada columna, pero no se ofrece un selector dedicado.
- **Soporte de múltiples idiomas:** el análisis sugería evolucionar el sistema con localización. La entrega final está únicamente en castellano para mantener el foco en los requisitos obligatorios.

4.2. Funcionalidades nuevas añadidas

- **Interfaz gráfica JavaFX** con dos ventanas (login y vista principal), tabla con todas las columnas, formulario de edición y barra de filtros.
- **Sistema de inicio de sesión (AuthService)** con cuenta admin / admin por defecto y registro de usuarios desde la propia interfaz JavaFX (enlace «Crear usuario» en la pantalla de login). Se eliminó la antigua cuenta de demostración juan / 1234.
- **Persistencia local de usuarios** en `~/ .agenda-tareas/users.csv`, de modo que las cuentas creadas siguen disponibles tras reiniciar la aplicación.
- **Persistencia en fichero (TaskRepository)** que guarda y restaura las tareas en formato CSV, satisfaciendo el criterio de gestión de archivos.
- **Aislamiento de tareas por usuario (v1.1):** nuevo campo *owner* en Task, octava columna en `tasks.csv`, método `TaskManager.getTasksByOwner(username)` y filtrado de toda la interfaz por el usuario autenticado. Se mantiene compatibilidad con ficheros antiguos sin la columna *owner*.
- **Cierre de sesión sin reiniciar la aplicación (v1.1):** nuevo botón **Cerrar sesión** en la cabecera de *MainView* que vuelve a mostrar *LoginView* en la misma ventana mediante un callback de `logout` pasado por constructor.
- **Clase lanzadora Launcher** que evita el error «faltan los componentes de JavaFX runtime» al ejecutar el proyecto desde IntelliJ IDEA sin configurar manualmente módulos.
- **Cobertura con pruebas unitarias JUnit 5** para *AuthService* (6 pruebas), *TaskManager* (16 pruebas) y *TaskRepository* (4 pruebas), incluyendo casos de ida y vuelta de serialización, persistencia de usuarios, aislamiento por propietario y compatibilidad con ficheros antiguos.
- **JavaDoc completo** en todas las clases públicas y miembros públicos, generable con `mvn javadoc:javadoc`.

- **Constructor secundario** en Task que permite recuperar el identificador desde fichero sin romper la unicidad, y constructor adicional con *owner* para etiquetar tareas con su propietario.
- Copia defensiva ordenada en *getAllTasks()* y método *findById()* para facilitar la integración con la interfaz gráfica.

5. Pruebas unitarias y verificación

El proyecto incluye pruebas unitarias con JUnit 5 que cubren los flujos principales de la lógica de negocio. Para ejecutarlas:

```
mvn test
```

Las pruebas verifican:

- Que el login del administrador por defecto (admin / admin) funciona y que la antigua cuenta juan / 1234 ya no existe.
- Que *register* crea usuarios nuevos, rechaza duplicados y valores en blanco o nulos, y que los usuarios creados se persisten en users.csv y se recuperan al reinstanciar el servicio.
- Que *addTask* añade tareas válidas y rechaza fechas inválidas.
- Que *removeTask* y *updateTask* funcionan correctamente cuando la tarea existe y devuelven false en caso contrario.
- Que *searchTask* es insensible a mayúsculas y busca en título y descripción.
- Que *filterByStatus*, *filterByPriority* y *filterByDates* aplican los criterios correctamente.
- Que *changeStatus* modifica el estado sólo cuando la tarea existe.
- Que *getAllTasks* devuelve una copia ordenada por ID.
- Que *getTasksByOwner* devuelve únicamente las tareas del usuario indicado y aísla a los distintos usuarios entre sí (caso de uso de la incidencia que motivó la v1.1).
- Que las tareas legadas sin propietario se cuentan como del administrador y no se muestran a usuarios nuevos como *juan*.
- Que *countByOwner* contabiliza correctamente las tareas por usuario.
- Que *TaskRepository* realiza correctamente la ida y vuelta de serialización aun con caracteres especiales (;) en el texto.
- Que *TaskRepository.loadAll()* devuelve una lista vacía si el fichero no existe.
- Que la columna *owner* se serializa y deserializa correctamente, y que los ficheros antiguos de siete columnas se cargan sin error.

Resultado de la última ejecución: 26 pruebas superadas, 0 fallos, 0 errores (entorno Linux, OpenJDK 21, Maven 3.9.x).

6. Estructura del proyecto

El proyecto sigue la estructura estándar de un proyecto Maven, lo cual permite que IntelliJ IDEA lo importe directamente sin configuración adicional:

```
agenda-tareas-javafx/  
|-- pom.xml  
|-- README.md  
|-- docs/  
| `-- Entrega-Final-Agenda-Tareas.pdf  
|-- src/main/java/com/juaniglesias/agendatareas/  
| |-- model/ (Task, Priority, Status)
```

```
| |-- manager/ (TaskManager)
| |-- persistence/ (TaskRepository)
| |-- auth/ (AuthService -- login + registro persistente)
| |-- ui/ (AgendaApp, LoginView, MainView)
| `-- Launcher.java (clase lanzadora JavaFX)
|-- src/test/java/com/juaniglesias/agendatareas/
|-- auth/ (AuthServiceTest)
|-- manager/ (TaskManagerTest)
|-- persistence/ (TaskRepositoryTest)
```